

TeleHubX

自动化任务说明书

Telegram 多账号广告 + 养号 + 智能客服 SaaS 系统

文档版本: v1.0

生成日期: 2026-05-01

共 22 类任务 + 4 类组合配套

本文档涵盖任务调度系统中 22 个原子任务类型 + 4 个组合配套,
每条任务包含: 用途、参数、执行步骤、预计时长、频率建议。

一、系统总览

TeleHubX 任务调度系统采用「服务端编排 + 客户端执行」架构：

- ☐ 服务端 (NestJS, 端口 9800) — 任务排队、状态机、租户隔离、API 入口
- ☐ 客户端 Agent (GramJS) — 每 15 秒拉取 dispatch, 按 BehaviorSimulator 模拟真人执行
- ☐ 所有时间间隔走 Gaussian 分布 (不是固定计时器), 避开 TG 风控自动化检测
- ☐ FloodWait 错误自动隔离账号 (quarantineUntil), 到期前不再领新任务

22 个任务按用途分为 8 大类：

- ☐ 组合配套 (4) — 一键启动多日剧本
- ☐ 群组发现 & 加入 (4) — 主动扩大社交图谱
- ☐ 自建群 (2) — 内部沙盒 / 剧本舞台
- ☐ 群组活动 (3) — 冒泡 / 对话剧本
- ☐ 拉新引流 (2) — 关键词 pipeline / 群成员爬取
- ☐ 触达 (2) — 加联系人 / 单条群发
- ☐ 内容输出 (4) — 频道发布 / 语音 / 图片 / 视频
- ☐ 互动信号 / 保活 (4) — Reaction / 浏览 / 资料 / 挂机

反检测铁律

- ☐ 一号一固定 IP, 绑定后不轮换
- ☐ 广告号与客服号 IP 段不重叠
- ☐ 变体文案相似度 < 70%
- ☐ ChatScript 一天一群一次
- ☐ 加别人群 ≤ 2 /天/账号
- ☐ 所有间隔 Gaussian 随机, 不用固定计时
- ☐ FloodWait 自动指数退避 + 账号隔离

二、任务详细说明（22 类）

1. 组合配套（一键启动多日剧本）

适合首次使用：选号 + 选套餐 → 按"立即执行"，系统按预设节奏自动跑完整个周期。

□ 一键托管 14 天

PRESET_FULL_14D

组合配套

用途：新号最省心方案。前 7 天自动养号（P0→P4），后 7 天运营热身爬坡。结束自动停止。

参数（payload）：

{ accountId: string }

执行步骤：

1. 系统自动展开为 PRESET_WARMUP_7D + PRESET_RAMPUP_7D 两条子任务串行排队
2. Day 1-7：执行 P0→P4 渐进养号（详见 PRESET_WARMUP_7D）
3. Day 8-14：执行运营热身（详见 PRESET_RAMPUP_7D）
4. Day 15 自动 transition 到 mature_ops 状态（需要手动启动 PRESET_MATURE_OPS）

预计时长：14 天

频率建议：一个号一生只跑 1 次（注册后立即启动）

□ 自动养号 7 天（Day 1-7）

PRESET_WARMUP_7D

组合配套

用途：P0→P4 渐进养号：从安静观察到逐步社交建立，让 TG 把账号视为正常用户。

参数（payload）：

{ accountId: string, intensity?: "mild" | "normal" }

执行步骤：

1. Day 1（P0）：账号注册后第一天 — 仅 IDLE_KEEPALIVE 心跳，0 任何外发动作，让账号安静"沉淀"
2. Day 2（P1）：BROWSE_CHANNEL × 2 个推荐频道（教育/娱乐/科技类），每个停 30-60s
3. Day 3（P2）：JOIN_CHANNELS 关注 1-2 个公开频道 + REACTION_BOOST 给 3-5 条消息加 □
4. Day 4-5（P3）：JOIN_GROUPS 加入 1 个公共讨论群 + GROUP_BUBBLE 在群里发 1-2 条短句（"了解" "□"）
5. Day 6-7（P4）：增加到 PROFILE_UPDATE（设头像/bio）+ BROWSE_CHANNEL 多频道 + 偶尔回复
6. 所有动作间隔 Gaussian（5min - 4h），每天总活动 < 30 分钟

预计时长：7 天

频率建议：一个号只跑 1 次（注册后第一周）

□ 运营热身 7 天（Day 8-14）

PRESET_RAMPUP_7D

组合配套

用途：从养号过渡到正式运营 — 活跃度爬坡，逐步开放对外触达。

参数（payload）：

{ accountId: string, intensity?: "mild" | "aggressive" }

执行步骤：

1. Day 8-9：JOIN_GROUPS_BY_KEYWORD 关键词搜群，每天加 1-2 个目标群
2. Day 10-11：开始 GROUP_BUBBLE 在已加入群中冒泡 + REACTION_BOOST
3. Day 12-13：开始 CONTACT_ADD 加 1-3 个联系人（mild 模式跳过此步）
4. Day 14：可选 CAMPAIGN_SINGLE 试发 5-10 条单条消息（aggressive 才启用）
5. 全程 Health Score 监控：< 60 自动停止当天剩余任务

预计时长：7 天

频率建议：紧接 PRESET_WARMUP_7D 之后跑

□ 成熟运营 Day 15+

PRESET_MATURE_OPS

组合配套

用途: Day 15+ 持续运行, 账号已"成熟", 可以正常承担广告/客服任务。

参数 (payload):

```
{ accountId: string }
```

执行步骤:

1. 每日 IDLE_KEEPALIVE 维持在线
2. 每日 1-2 次 BROWSE_CHANNEL + REACTION_BOOST 维持互动信号
3. 每日 GROUP_BUBBLE 在自有群冒泡
4. 其余任务 (CAMPAIGN_SINGLE / CONTACT_ADD / 等) 由租户手动派发
5. 不限期, 直到账号被暂停或删除

预计时长: 不限期

频率建议: 常驻运行

2. 群组发现 & 加入

□ 自动加群 (邀请链接)

JOIN_GROUPS

群组发现

用途: 通过邀请链接 / 群 chat_id 列表批量加群。

参数 (payload):

```
{ inviteLinks?: string[], chatIds?: string[], inviteIntervalSec?: [60,180] }
```

执行步骤:

1. 遍历 inviteLinks 或 chatIds 数组
2. 对每条 invite link: 调用 messages.ImportChatInvite(hash)
3. 对每条 @username / chatId: 调用 channels.JoinChannel(entity)
4. USER_ALREADY_PARTICIPANT 视为成功跳过
5. 每加完一个群, Gaussian 间隔 60-180 秒后再加下一个
6. 最多 2 群/天/账号 (超过自动暂停剩余)

预计时长: 取决于数量, 平均 2 分钟/群

频率建议: ≤ 2/天/账号

□ 关键词搜群+加

JOIN_GROUPS_BY_KEYWORD

群组发现

用途: 按关键词搜索 TG 公开群, 按成员数过滤后加入 (仅加, 不爬成员)。

参数 (payload):

```
{ keywords: string[], minMembers?: 100, maxPerDay?: 3 }
```

执行步骤:

1. 对每个关键词调用 contacts.Search(q, limit=30)
2. 过滤: is_chat = true, members >= minMembers, 不在已加入列表
3. 按成员数降序排列, 取前 maxPerDay 个
4. 逐个 messages.ImportChatInvite 或 channels.JoinChannel
5. 每加一个群间隔 5-15 分钟
6. 记录到 LeadCandidate 表 (来源标注 "keyword:外汇")

预计时长: 每个关键词 ~10 分钟 (含间隔)

频率建议: ≤ 3/天/账号, 建议 1-2 关键词/天

□ Follow 频道

JOIN_CHANNELS

群组发现

用途: 关注公开频道 (不是群组)。频道是单向广播, 加入风险极低。

参数 (payload):

```
{ channels: string[] } // 数组每条可以是 @username 或 https://t.me/joinchat/xxx
```

执行步骤:

1. 对邀请链接 (https://t.me/+xxx): messages.ImportChatInvite(hash)
2. 对 @username: getEntity → channels.JoinChannel
3. 加群间隔 Gaussian 60-180 秒

4. USER_ALREADY_PARTICIPANT 视为成功

预计时长：每个频道 ~2 分钟

频率建议：养号期推荐每天 1-2 个，老号无限制

□ 接受群组邀请

ACCEPT_INVITES

群组发现

用途：接受所有 pending 状态的群组邀请。

参数 (payload)：

```
{ autoAcceptAll: boolean }
```

执行步骤：

1. 调用 messages.GetDialogFilters 获取所有未读对话
2. 过滤 dialog.peer.invitation_pending = true
3. 逐个调用 messages.HideChatJoinRequest(approved=true)
4. 每接受一个间隔 30-90 秒

预计时长：< 5 分钟

频率建议：建议每周跑 1 次

3. 自建群（内部沙盒）

□ 自建测试群

GROUP_CREATE

自建群

用途：系统自动委派一个执行组内的账号去新建群组，用作 ChatScript 剧本舞台或客户沉淀池。

参数 (payload)：

```
{ title: string, type: "small" | "mega", initialMemberAccountIds: string[] }
```

执行步骤：

1. 从执行组中挑选一个 P3+ 健康度 ≥ 80 的账号担任“创建者”
2. 调用 messages.CreateChat(title, users=[初始成员]) 或

channels.CreateChannel(megagroup=true)

3. 建群成功后保存 tgChatId 到 GroupRegistry 表（标注 owner_account_id）
4. 可选：群头像、群描述、置顶规则一并设置
5. 间隔 ≥ 24 小时才能再建一个（同账号）

预计时长：~3 分钟

频率建议： ≤ 1 /账号/24h

□ 邀请账号入群

GROUP_INVITE_MEMBERS

自建群

用途：让某个账号 (inviter) 邀请同执行组的其他账号进入指定群。常用于刚建好的自建群拉满人。

参数 (payload)：

```
{ tgChatId: string, inviterAccountId: string, targetAccountIds: string[] }
```

执行步骤：

1. 由 inviter 调用 channels.InviteToChannel(channel, users=targetAccountIds)
2. 若失败 (USER_PRIVACY_RESTRICTED) 改用 messages.AddChatUser 单条加
3. 每邀请一个间隔 60-300 秒
4. 邀请总数 ≤ 6 人/次（避免触发 too many invitations）

预计时长：取决于数量

频率建议：建群后立即跑 1 次

4. 群组活动

□ 群内冒泡

GROUP_BUBBLE

群组活动

用途：在指定群发短句/emoji（"了解" "收到"）维持账号活跃感。

参数 (payload)：

```
{ tgChatId: string, count?: [1,2], textPool?: string[] }
```

执行步骤：

1. 从默认池随机取（"了解" "收到" "嗯嗯" "好的" 等 14 项）
2. 通过 sendMessageLikeHuman：先发 typing 状态 → 等 60-100ms × 字数 → SendMessage
3. 冒泡间隔 5-30 分钟（不要连刷）
4. 群内有真人活跃时跳过（防止打断真人对话）

预计时长：每条 ~5-30 分钟

频率建议：每日 1-3 次/群，每群每天最多 6 次

□ A+B 双角色剧本

CHAT_SCRIPT_AB

群组活动

用途：在群里用两个号演一段对白：A 提问 → B 回答，模拟"群成员真实讨论"。

参数 (payload)：

```
{ tgChatId, scriptId, accountAId, accountBId }
```

执行步骤：

1. 从 ScriptLibrary 加载剧本（每段 6-12 轮对话）
2. 账号 A 切换上下文，发第 1 句（typing → send，含 60-120s 思考间隔）
3. 账号 B 收到推送后切换，回第 2 句
4. 交替直到剧本跑完
5. 群内有真人活跃时立即停（人不能干扰人，两个 bot 自演被发现就破功）
6. 一群一天最多 1 次

预计时长：15-40 分钟

频率建议：≤ 1/群/天

□ 4 人剧本

CHAT_SCRIPT_4P

群组活动

用途：4 个账号在群里演一段更复杂的"多方讨论"，比 AB 更真实。

参数 (payload)：

```
{ tgChatId, scriptId, accountIds: [4 个 id] }
```

执行步骤：

1. 加载 4 角色剧本
2. 按剧本里的 actor 字段切换账号
3. 每句之间 30-180 秒思考时间
4. 同样：群内有真人活跃立即停
5. 总时长一般 30-60 分钟

预计时长：30-60 分钟

频率建议：≤ 1/群/3 天

5. 拉新引流

□ 关键词智能引流

KEYWORD_LEAD_HUNT

拉新引流

长任务 30 天

用途：4 阶段 pipeline：搜群 → 加群 → 等沉淀 → 爬成员 → 可选触达。默认 30 天，全自动。

参数 (payload)：

```
{ keywords: [], maxGroupsPerDay, scrapeDelayHours, maxOutreachPerDay, durationDays:30 }
```

执行步骤：

1. 阶段 1 (Day 1-7) — 搜群+加群：每天用关键词搜 contacts.Search → 加 maxGroupsPerDay（默认 2）个群
2. 阶段 2 (Day 8-9) — 沉淀等待：scrapeDelayHours 小时（默认 48h），让账号在群里"自然"待几天
3. 阶段 3 (Day 10-25) — 爬成员：channels.GetParticipants 抓取群成员（每群最多 50 人），写

入 LeadCandidate 表

4. 阶段 4 (Day 26-30, 可选) — 触达: 每天 maxOutreachPerDay (默认 5) 条消息加 contact, 模板从 KB 取
5. 全程 Health Score 监控, < 50 自动暂停整个 pipeline
6. 中途可以 PATCH 任务暂停 / 调整参数

预计时长: 30 天 (可调)

频率建议: 每个广告号最多 1 个并行 lead_hunt

□ 群成员爬取

GROUP_SCRAPE

拉新引流

用途: 独立运行 (不带搜群), 针对已加入的群直接抓取成员名单。

参数 (payload):

```
{ tgChatIds: string[], maxScrapePerGroup?: 50 }
```

执行步骤:

1. 对每个 tgChatId: getEntity → channels.GetParticipants(filter=Recent, limit=200)
2. 过滤: is_bot=false, deleted=false, last_seen 在 30 天内
3. 取前 maxScrapePerGroup 个写入 LeadCandidate (status=raw, source="scrape:群名")
4. 每群之间 Gaussian 间隔 10-30 分钟

预计时长: 每群 ~2 分钟 (含间隔 10-30 分钟)

频率建议: 同群 ≥ 7 天才能再爬一次

6. 触达

□ 加 contact

CONTACT_ADD

触达

高风险

用途: 把目标用户加入通讯录 (前置: private chat 才能稳定到达, 否则会被 spam folder 吃掉)。

参数 (payload):

```
{ mode: "username" | "phone", targets: string[], maxPerDay: number }
```

执行步骤:

1. 对 username 模式: getEntity(@user) → contacts.AddContact
2. 对 phone 模式: contacts.ImportContacts([[phone, first_name]])
3. 每加一个 Gaussian 间隔 3-10 分钟
4. 失败 (USER_PRIVACY_RESTRICTED / PEER_FLOOD) 立即标记账号 cooldown 6 小时
5. maxPerDay 上限: 新号 (Day < 14) 5 个, 老号 20 个

预计时长: 取决于数量

频率建议: 严格按 maxPerDay; FloodWait 后强制冷却

□ 单条消息 (campaign)

CAMPAIGN_SINGLE

触达

高风险

用途: 向目标列表批量发消息 (campaign 4 层架构的最小执行单元)。AI Variant 生成 N 种文案变体。

参数 (payload):

```
{ targets: string[], variants: string[], intervalSec: [60, 300] }
```

执行步骤:

1. 对每个 target 随机抽一个 variant
2. sendMessageLikeHuman: typing → 60-100ms × 字数 → SendMessage
3. 每条间隔 Gaussian 60-300 秒
4. 300/30 配额规则: 30 天内最多 300 条 (即 ~10 条/天)
5. 中途 FloodWait → 立即停止 + 账号隔离 + 任务标记 failed
6. 中途 PEER_FLOOD → 暂停 6h 重试

预计时长: 取决于数量; 100 条 ≈ 4 小时

频率建议: ≤ 10/天/号 (300/30 规则)

7. 内容输出

□ 发频道 / Story

POST_CHANNEL

内容输出

用途：在自有频道发一条新消息或 Story。

参数 (payload)：

```
{ channelId: string, content: string, mediaAssetId?: string }
```

执行步骤：

1. getEntity(channelId)
2. 若 mediaAssetId: 从 Asset 表取文件 → upload → SendMedia
3. 否则 SendMessage(content)
4. 可选 schedule_date 字段做定时发布

预计时长：< 1 分钟

频率建议：建议每个频道每天 1-3 条

□ 发语音（素材池随机）

MEDIA_VOICE

内容输出

用途：从素材库 voice 类别随机抽一条语音，发到群/频道。

参数 (payload)：

```
{ targetType: "group" | "channel", targetId: string, assetCategory: "voice" }
```

执行步骤：

1. 从 Asset 表 SELECT WHERE category=voice ORDER BY RANDOM() LIMIT 1
2. upload 文件到 TG (uploadFile)
3. SendMedia 到 targetId
4. 发完标记该 asset usage_count++

预计时长：< 2 分钟

频率建议：每个号每天 ≤ 5 条媒体

□ 发图片（素材池随机）

MEDIA_PHOTO

内容输出

用途：从素材库 photo 类别随机抽一张图片，发到群/频道。

参数 (payload)：

```
{ targetType, targetId, assetCategory: "photo" }
```

执行步骤：

1. 同 MEDIA_VOICE, category=photo
2. 可选 caption 字段（从 KB 抽匹配的文案）
3. AI Variant: 图片做 ±2 像素微偏移（避免 hash 重复）

预计时长：< 1 分钟

频率建议：同上

□ 发视频（素材池随机）

MEDIA_VIDEO

内容输出

用途：从素材库 video 类别随机抽一条视频。

参数 (payload)：

```
{ targetType, targetId, assetCategory: "video" }
```

执行步骤：

1. 同 MEDIA_VOICE, category=video
2. 上传后 SendMedia
3. 可选附 caption + thumb

预计时长：取决于文件大小

频率建议：同上

8. 互动信号 / 保活

□ 给消息加 Reaction

REACTION_BOOST

互动 / 保活

用途：给频道/群最近的消息加 emoji reaction，模拟"路过点个赞"。

参数 (payload)：

```
{ tgChatId, count?: [3,8], emojiPool?: ["👍" "👎" "😂" "🤔" "🙄" ] }
```

执行步骤：

1. getEntity(tgChatId) → getMessages(limit=50)
2. 随机洗牌取 count 条（默认 3-8）
3. 对每条调用 messages.SendReaction(emoji 池随机抽)
4. 每条 reaction 间隔 5-30 秒
5. 消息不允许 reaction 自动跳过

预计时长：~5 分钟

频率建议：每个号每天 1-2 次，每群最多 1 次/天

□ 浏览频道（模拟阅读）

BROWSE_CHANNEL

互动 / 保活

用途：打开频道、拉历史消息、停留 N 秒像真人在看。最重要的"低风险信号"任务。

参数 (payload)：

```
{ channels: string[], readDurationSec?: [20,90] }
```

执行步骤：

1. 对每个频道：getEntity → getMessages(limit=20)
2. 停留 readDurationSec 秒（默认 20-90s Gaussian）
3. 在 TG 后台日志里这表现为"打开聊天 + 滚动浏览"
4. 不发消息、不点 reaction，纯被动

预计时长：~每个频道 1-2 分钟

频率建议：每个号每天 2-5 次（养号期最重要任务）

□ 更新资料（bio / first_name / 头像）

PROFILE_UPDATE

互动 / 保活

用途：更新账号资料 — 真实用户每隔几个月会改一次，长期不动反而异常。

参数 (payload)：

```
{ firstName?, lastName?, bio?, photoAssetId? }
```

执行步骤：

1. 若有 photoAssetId: upload → photos.UploadProfilePhoto
2. 若有 bio / firstName / lastName: account.UpdateProfile
3. 一次只改一个字段，每个改动间隔 30-60 秒

预计时长：< 3 分钟

频率建议：建议每个号每月 1 次（小幅改）

□ 挂机保活

IDLE_KEEPALIVE

互动 / 保活

用途：让账号在 TG 客户端列表显示在线，无任何副作用。最简单的健康信号。

参数 (payload)：

无

执行步骤：

1. 调用 account.UpdateStatus(offline: false)
2. 一次性立即返回
3. 通常由 KeepOnline 心跳服务每 30-60 秒自动跑（不需要手动派发）

预计时长：< 1 秒

频率建议：agent 自动每 30-60 秒跑（无需手动）

三、附录

A. 任务状态机

- ☐ pending — 已创建，等待 agent 领取 (scheduledAt 未到 / 还没拉到)
- ☐ running — agent 已认领，正在执行
- ☐ paused — 用户手动暂停 (仅 running 任务可暂停)
- ☐ done — 执行成功
- ☐ failed — 执行失败，errorMsg 字段记录原因

B. 任务复用「执行」按钮

任意任务 (包括 done/failed) 的"操作"列都有 [执行] 按钮:

- ☐ 点击后 POST /tasks/:id/run-now
- ☐ 系统克隆当前任务 (保留所有 payload + accountId)
- ☐ 新任务的 scheduledAt = now, 状态 = pending
- ☐ 原任务保留作历史记录, 不修改
- ☐ 适用场景: 补发、复测、重跑某个客户

C. 调度内核

- ☐ agent 每 15 秒调用 POST /tasks/dispatch (accountIds=[本机所有 online 账号], limit=5)
- ☐ 服务端原子事务: SELECT pending + scheduledAt<=now → UPDATE status=running
- ☐ 同一任务最多被一个 agent 领取 (避免重复执行)
- ☐ 执行完成后 PATCH /tasks/:id { status: done, finishedAt }
- ☐ FloodWait 异常 → 解析 wait_seconds → 写 quarantineUntil → 任务标 failed

D. BehaviorSimulator 关键参数

- ☐ gaussianDelayMs(min, max) — 用 Box-Muller 生成正态分布间隔 (μ =中点, σ =range/4)
- ☐ typingDurationMs(text) — 60-100ms × 字符数 + 高斯抖动
- ☐ sendMessageLikeHuman — 先发 messages.SetTyping → 等 typing 时长 → SendMessage
- ☐ simulateReading(min, max) — 单纯睡 min-max 秒
- ☐ interTaskDelay() — 两个任务之间 5-30 分钟

E. 任务运行环境

- ☐ TG 客户端库: GramJS (TypeScript) + StringSession
- ☐ 代理: 每号一固定 SOCKS5 (HTTP 代理通过本地 bridge 转 SOCKS5)
- ☐ 设备指纹: 每号独立 (Samsung/Xiaomi/OPPO/Vivo 等 19 机型 × 3 Android 版本 × 9 TG 版本)
- ☐ 运行进程: apps/agent (Node 20) — pm2 守护, DB 驱动启动
- ☐ 后端: apps/server (NestJS, 端口 9800) — 任务 CRUD + dispatch + 状态聚合